

Data Pipelines, Kafka & Code Quality

Study Guide for Sprint 7

1. Topic Overview

Topic	Focus
Batch vs. real-time	Choose based on boundedness, deadlines, and freshness.
Event-driven architecture	Use events when independent consumers react to something that happened.
Kafka partitioning	Choose keys from ordering requirements.
Consumer groups	Use independent groups for independent consumption progress.
Offsets & idempotency	Design for crash recovery, retries, and possible reprocessing.
Data quality	Make dropped and invalid records visible.
Code quality	Separate responsibilities and remove hidden dependencies.
SonarQube	Fix root causes and rerun analysis.
Secret scanning	Git history can retain removed credentials.
GenAI review	Verify AI suggestions against actual evidence.

2. Question-by-Question Review

Question 1

A CSV contains 100 records, but a batch job reports processing only 96 with no errors. What should concern you most?

Study point — Silent data loss: A lower processed count with no explanation can indicate records are being dropped without visibility.

Exam cue: input count $\neq$ processed count $\rightarrow$ investigate

Question 2

A nightly billing job processes one complete day's transactions before 5 AM. Which processing style best fits this requirement?

Study point — Batch vs. real-time: A bounded data set with a fixed completion deadline is naturally suited to batch processing.

Exam cue: bounded work + deadline

Question 3

A fraud system currently evaluates card transactions immediately. What risk comes from changing it to run every 30 minutes?

Study point — Freshness: Batching introduces delay, so important activity may not be evaluated until the next scheduled run.

Exam cue: real-time need → stale data risk

Question 4

A developer says event-driven systems are simply scheduled batch jobs. What key distinction are they missing?

Study point — Event vs. request-driven: Event/request style describes interaction between components; batch/real-time describes processing timing.

Exam cue: different architectural axes

Question 5

Inventory, analytics, and notifications must independently react after an order is placed. What architecture supports this well?

Study point — Event-driven fan-out: Publishing an event allows multiple independent consumers to react without blocking the original operation.

Exam cue: many reactions → publish event

Question 6

Kafka must preserve the order of all transactions for the same customer. Why is a random transaction ID a poor partition key?

Study point — Kafka partition keys: Random keys can distribute one customer's events across partitions, losing the required per-customer ordering.

Exam cue: partition key follows ordering need

Question 7

A Kafka consumer processes a message and crashes before committing its offset. What should you expect after restart?

Study point — Kafka offsets: The broker knows committed progress, not merely work completed in memory, so processing may repeat.

Exam cue: processed ≠ committed

Question 8

A validator reports zero bad records even though invalid records are known to exist. What does this tell you?

Study point — Validation quality: A quiet validation report proves little if the validation rules themselves are ineffective.

Exam cue: no errors reported ≠ correct

Question 9

An extracted method still updates shared static totals. Why is it difficult to unit test in isolation?

Study point — Hidden state: Its behavior depends on state outside its explicit inputs and outputs.

Exam cue: shared state reduces isolation

Question 10

A dashboard creates a new Kafka consumer group and replays years of events on every refresh. What scalability problem results?

Study point — Bounding Kafka reads: The amount of work grows with retained topic history unless the read is bounded by time or offsets.

Exam cue: unbounded replay grows over time

Question 11

Billing and fraud both consume the same Kafka topic independently. Why should they normally use separate consumer groups?

Study point — Consumer groups: Separate groups maintain independent offsets and allow each application to consume the topic at its own pace.

Exam cue: group = independent progress

Question 12

A consumer may receive the same message again after a crash. What property should its processing logic have?

Study point — Idempotency: Repeated processing should not create unintended duplicate effects.

Exam cue: retry should not duplicate effect

Question 13

Two consumer groups read the same Kafka topic. Does one group's progress change the other's position?

Study point — Kafka retention and groups: Consumer groups track their offsets independently while messages remain available according to retention.

Exam cue: groups do not share progress

Question 14

Before using GenAI to diagnose a Kafka exception, what should you do with the stack trace?

Study point — GenAI debugging: Read the complete trace and causal chain yourself so the AI explanation can be checked against actual evidence.

Exam cue: understand evidence before AI fix

Question 15

Rerunning a failed import inserts the same invoices twice. What design property is missing?

Study point — Idempotent batch loads: A safe rerun should not create duplicate business effects; uniqueness and upsert strategies can help enforce this.

Exam cue: rerun should be safe

Question 16

One method parses files, calculates fees, updates totals, and formats output. What design smell should you recognize?

Study point — Single Responsibility: The method contains multiple independent responsibilities, making maintenance and isolated testing harder.

Exam cue: many jobs → SRP smell

Question 17

SonarQube reports that a file reader may remain open. What Java feature should you consider?

Study point — Resource management: Resource lifecycle should be handled reliably, including exceptional paths.

Exam cue: resource cleanup must be guaranteed

Question 18

A SonarQube fix causes a different quality issue to appear. What should the team do next?

Study point — Quality-gate iteration: Review the new finding on its merits and continue the fix-and-rerun cycle rather than hiding it.

Exam cue: fix → rerun → inspect

Question 19

An API key was removed from current code after being committed to Git. Why can the credential still be exposed?

Study point — Secret scanning: Earlier commits can retain the secret even when the working tree is clean.

Exam cue: current code clean ≠ history clean

Question 20

A nightly process was adequate, but a new consumer now needs updates within seconds. Why should the architecture be reconsidered?

Study point — Changing requirements: Architecture choices should be revisited when latency, volume, or consumer requirements materially change.

Exam cue: changed constraint → reconsider design

3. Final Review Checklist

- ☐ Choose batch or real-time from latency and boundedness requirements.
- ☐ Distinguish event/request interaction from batch/real-time timing.
- ☐ Choose Kafka partition keys from ordering requirements.
- ☐ Explain independent consumer groups and committed offsets.
- ☐ Explain why Kafka consumers and batch jobs often need idempotency.
- ☐ Recognize silent data loss and ineffective validation.
- ☐ Recognize shared state and Single Responsibility problems.
- ☐ Explain why unbounded Kafka replay scales poorly.
- ☐ Respond appropriately to SonarQube findings.
- ☐ Explain why Git history can preserve leaked secrets.
- ☐ Verify GenAI debugging suggestions against evidence.
- ☐ Revisit architecture when requirements materially change.

4. One-Minute Memory Map

Processing: Batch fits bounded/deadline work; real-time fits continuous freshness.

Kafka: Partition key controls ordering boundaries; consumer groups track progress independently; committed offsets drive restart behavior.

Reliability: Expose bad data, bound reads, and make retries idempotent.

Quality & security: Fix static-analysis findings at the cause, verify AI suggestions, and rotate credentials exposed in Git.